

Running FlightPath locally (Windows)

One-command path to get the FlightPath Flutter app booting in an Android emulator on the Windows dev PC.

iOS builds live on the Mac only. This doc is Windows-first. For iOS / TestFlight, see the Mac-side workflow (not covered here).

Prerequisites (one-time)

1. **Flutter SDK** (`^3.6.0` , matches `pubspec.yaml`) on PATH.
 - <https://docs.flutter.dev/get-started/install/windows>
 - Sanity check: `flutter --version`
2. **Android Studio** with at least one AVD (virtual device) created.
 - Android Studio -> More Actions -> Virtual Device Manager -> Create Device
 - Any Pixel profile + API 34 image is fine.
3. **Flutter Android toolchain** green: `flutter doctor` shows no red X on "Android toolchain" or "Android Studio".
4. **Firebase config is already in the repo** (`android/app/google-services.json` , `lib/firebase_options.dart`). Don't touch these.

First-time setup

```
cd D:\ed-sync\Projects\Apps\flightpath
copy tool\.env.local.example tool\.env.local
```

Open `tool\.env.local` and fill in any real keys you have. For day-to-day dev you can leave everything blank — the run script drops in placeholder `dart-defines` so the app still boots in free-tier mode.

When you get real RevenueCat keys (pending in the RevenueCat dashboard), drop `REVENUECAT_ANDROID_KEY=goog_...` into `tool\.env.local` and re-run. The file is gitignored via `.env.*` in `.gitignore`.

Running

Option A - PowerShell one-liner

```
.\tool\run-emulator.ps1
```

The script will:

1. Check `flutter` is on PATH.
2. Load `tool\.env.local` if present.
3. Detect a running Android device/emulator, or auto-launch the first AVD it finds (`flutter emulators --launch <name>`) and wait up to 90s for it to come online.
4. Run `flutter run --debug` with the right `--dart-define` flags.

Flags:

- `-Profile` profile build

- `-Release` release build (uses debug signing)
- `-Emulator "Pixel_7_API_34"` pick a specific AVD
- `-NoLaunchEmulator` fail instead of auto-starting an AVD

Option B - VS Code F5

Open the repo in VS Code, make sure an emulator is running (or let the PowerShell script launch one once), then F5 - > pick **"FlightPath - Android emulator (debug)"**. Hot reload works out of the box.

One-time setup: the `.vscode/launch.json` file isn't committed (the setup tool couldn't write into `.vscode/` on this machine). Create it manually at `D:\ed-sync\Projects\Apps\flightpath\.vscode\launch.json` with this content:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "FlightPath - Android emulator (debug)",
      "request": "launch",
      "type": "dart",
      "program": "lib/main.dart",
      "flutterMode": "debug",
      "toolArgs": [
        "--dart-define=REVENUECAT_ANDROID_KEY=goog_dev_placeholder",
        "--dart-define=REVENUECAT_IOS_KEY=appl_dev_placeholder"
      ]
    },
    {
      "name": "FlightPath - Android emulator (profile)",
      "request": "launch",
      "type": "dart",
      "program": "lib/main.dart",
      "flutterMode": "profile",
      "toolArgs": [
        "--dart-define=REVENUECAT_ANDROID_KEY=goog_dev_placeholder",
        "--dart-define=REVENUECAT_IOS_KEY=appl_dev_placeholder"
      ]
    },
    {
      "name": "FlightPath - Android emulator (release)",
      "request": "launch",
      "type": "dart",
      "program": "lib/main.dart",
      "flutterMode": "release",
      "toolArgs": [
        "--dart-define=REVENUECAT_ANDROID_KEY=goog_dev_placeholder",
        "--dart-define=REVENUECAT_IOS_KEY=appl_dev_placeholder"
      ]
    }
  ]
}
```

Once real RevenueCat keys land, swap the placeholder values in `toolArgs` for the real ones (or just use the PowerShell script, which picks up `tool/.env.local` automatically).

Dart-defines the app needs

Flag	Required?	Source	Placeholder fallback
REVENUECAT_ANDROID_KEY	yes (debug build throws <code>StateError</code> if missing)	<code>tool/.env.local</code>	<code>goog_dev_placeholder</code>
REVENUECAT_IOS_KEY	no on Windows	<code>tool/.env.local</code>	<code>appl_dev_placeholder</code>

Both are read via `String.fromEnvironment` in `lib/shared/services/subscription_service.dart`. The debug build will throw loudly if `REVENUECAT_ANDROID_KEY` is completely empty — that's why the script always supplies at least a placeholder string. With the placeholder, `Purchases.configure` fails, the init retries 3x, and the app continues in free-tier mode. Paywall screens will show but won't display real offerings until a real key lands.

Troubleshooting

"flutter is not on PATH" Add the Flutter SDK `bin` directory to your Windows PATH and restart PowerShell / VS Code.

"No Android AVDs found" Open Android Studio -> More Actions -> Virtual Device Manager -> Create Device. Pick any Pixel profile + API 34. Re-run the script.

"Emulator did not come online within 90 seconds" Launch the AVD manually from Android Studio's Device Manager, wait until the home screen shows, then re-run `.\tool\run-emulator.ps1`.

"StateError: RevenueCat SDK key missing" Your `tool/.env.local` has `REVENUECAT_ANDROID_KEY=` with no value AND the script's placeholder isn't being applied. Make sure you're invoking the script (not raw `flutter run`). If you are, check the script output for a "Loading env" line — a malformed line in `.env.local` can clobber the fallback. Easiest fix: comment the line out with `#`.

Firebase errors at boot Confirm `android/app/google-services.json` and `lib/firebase_options.dart` exist. If they do, run `flutter clean && flutter pub get` once and retry.

Hot reload not firing VS Code: make sure you launched via F5 (not the PowerShell script). The script runs `flutter run` in the terminal, where hot reload works by pressing `r` in the terminal window.

Slow first build First `flutter run` on a fresh checkout pulls Gradle + Android build tools. 5-10 min is normal. Subsequent runs are seconds.

What NOT to do

- Do not commit `tool/.env.local`.
- Do not edit `lib/firebase_options.dart`, `google-services.json`, or `GoogleService-Info.plist` — they're generated by FlutterFire.
- Do not run release builds for distribution from Windows. Release APKs from here are only for local smoke-testing; Play Store bundles should be signed with a real keystore (not wired up yet).